Clasificación de imágenes de Pokémon mediante una CNN, un autoencoder convolucional y transferencia de aprendizaje con EfficientNetB0

Jacob Espejel Guiza [1]

Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Querétaro

[1] A01712021@tec.mx

Resumen

La clasificación visual de Pokémon requiere distinguir categorías con rasgos similares y reconocer una misma categoría bajo diferentes poses, fondos y estilos de ilustración. Este estudio examinó tres estrategias de aprendizaje de representaciones: una red neuronal convolucional con inicialización aleatoria, un autoencoder convolucional con un clasificador sobre su representación latente y EfficientNetB0 preentrenada en ImageNet. La metodología comprendió auditoría de imágenes y etiquetas, agrupación de variantes de una misma fuente, preparación de particiones, análisis exploratorio y entrenamiento con selección mediante validación. El conjunto empleado en el experimento final incluyó 6,989 imágenes de 150 categorías, distribuidas en 4,898 muestras de entrenamiento, 1,045 de validación y 1,046 de prueba. El autoencoder alcanzó una exactitud de validación de 6.79 % y un F1 macro de 4.15 %, lo que evidenció una capacidad discriminativa limitada de la representación aprendida mediante reconstrucción. En el experimento final de EfficientNetB0, el ajuste fino parcial aumentó la exactitud de validación de 85.93 % a 87.08 % respecto a la cabeza entrenada con el extractor congelado. El modelo seleccionado obtuvo en prueba una exactitud de 87.57 %, una exactitud top-3 de 95.12 % y un F1 macro de 87.24 %. El análisis de errores identificó dificultades en categorías visualmente próximas, especialmente Pidgeotto y Pidgeot. Los resultados indican que la transferencia de aprendizaje permitió obtener un clasificador eficaz para el conjunto estudiado. La interpretación comparativa se limita por la ejecución incompleta de la CNN, las diferencias de configuración entre experimentos y la evaluación con una única semilla.

Palabras clave: clasificación de imágenes, Pokémon, aprendizaje profundo, autoencoder convolucional, EfficientNetB0, transferencia de aprendizaje.

I. Introducción

La clasificación de imágenes consiste en asignar una categoría a partir de la información visual de una imagen. Aunque esta tarea puede parecer sencilla cuando las categorías tienen diferencias evidentes, su dificultad aumenta si comparten colores, siluetas o estructuras. También resulta necesario reconocer que imágenes de una misma categoría pueden variar considerablemente en posición, escala, fondo y estilo de representación. Un clasificador útil debe conservar las diferencias que identifican a cada clase y reducir la influencia de variaciones que no modifican su identidad.

Las imágenes de Pokémon permiten estudiar este problema en un dominio con numerosas categorías y semejanzas visuales. Algunos personajes pertenecen a una misma línea evolutiva y comparten rasgos de diseño; otros pueden confundirse por su forma o apariencia general. Al mismo tiempo, una categoría puede representarse mediante ilustraciones con poses y fondos distintos. Estas condiciones permiten analizar tanto la capacidad de reconocimiento de un modelo como los tipos de errores que comete.

El desempeño de una red depende de la representación que aprende de las imágenes. Una CNN entrenada desde cero debe desarrollar sus características visuales exclusivamente a partir de los ejemplos disponibles. Un autoencoder aprende a comprimir y reconstruir la imagen, de modo que su representación responde a

la conservación de información visual. La transferencia de aprendizaje reutiliza características previamente aprendidas en otro conjunto de datos y las adapta a una tarea nueva. Cada estrategia introduce una relación distinta entre datos, objetivo de entrenamiento y conocimiento previo.

En este estudio se implementaron estas tres estrategias y se evaluó un clasificador final basado en EfficientNetB0. Además de medir su exactitud, se analizaron la calidad de los datos, la evolución del entrenamiento, el rendimiento por categoría y las confusiones más frecuentes. Esta aproximación permite interpretar el resultado como parte de un procedimiento experimental completo, en el cual la preparación de los datos y el criterio de evaluación son tan relevantes como la arquitectura.

A. *Objetivo y alcance*

El objetivo fue desarrollar y evaluar un sistema de clasificación de imágenes de Pokémon en 150 categorías, considerando diferentes formas de aprender sus características visuales. En particular, se examinó la utilidad de una representación obtenida mediante reconstrucción y se cuantificó el desempeño de EfficientNetB0 con entrenamiento de cabeza y ajuste fino parcial.

La tarea se definió como clasificación de una imagen completa. Cada imagen se asoció con una sola etiqueta y el modelo produjo una distribución sobre las categorías conocidas. No se realizó localización de objetos ni segmentación. La evaluación principal se centró en el modelo final de EfficientNetB0; las otras configuraciones se utilizaron para examinar el comportamiento de las estrategias implementadas, con las restricciones experimentales descritas en la metodología.

II. Fundamento del enfoque

A. *Aprendizaje de características con una CNN*

Una red convolucional aprende a representar las imágenes mediante filtros que se aplican sobre distintas regiones. Durante el entrenamiento, sus parámetros se ajustan para que las respuestas de esos filtros contribuyan a distinguir las categorías. Las primeras transformaciones trabajan sobre información local, mientras que las capas posteriores combinan sus resultados en representaciones más amplias. Esto permite que la decisión dependa de patrones aprendidos, en lugar de reglas definidas manualmente para cada personaje.

Las operaciones de reducción espacial disminuyen el tamaño de los mapas de características y permiten procesar la imagen de manera progresivamente más compacta. Finalmente, un clasificador combina la representación obtenida y asigna un valor a cada categoría. La función softmax transforma esos valores en una distribución cuya suma es uno. La categoría con mayor valor constituye la predicción principal; las siguientes proporcionan alternativas ordenadas por la preferencia del modelo.

Entrenar desde cero significa que los filtros comienzan con valores aleatorios. Por tanto, tanto la extracción de características como la decisión final deben aprenderse con las imágenes del estudio. Esta condición constituye una referencia útil para examinar el aprendizaje supervisado, aunque su resultado depende de la cantidad de datos y del entrenamiento realizado.

B. *Representación latente mediante un autoencoder*

El autoencoder aprende a reconstruir su propia entrada. Para ello utiliza un codificador, o encoder, que transforma la imagen en una representación compacta, y un decodificador, o decoder, que intenta reconstruirla a partir de esa representación. El espacio intermedio se denomina latente: reúne la información que el modelo conserva para realizar la reconstrucción. El aprendizaje de representaciones compactas mediante este principio se ha estudiado en reducción de dimensionalidad (Hinton & Salakhutdinov, 2006).

En esta investigación, el interés del autoencoder fue utilizar el encoder como extractor de características. Una vez entrenado con reconstrucción, se mantuvo fijo y se conectó a un clasificador supervisado. De esta manera se examinó si la información necesaria para reconstruir las imágenes también permitía diferenciar sus categorías.

Ambas tareas requieren información visual, pero tienen objetivos distintos. La reconstrucción valora la semejanza de la imagen completa; la clasificación depende de conservar los rasgos que separan una categoría de

las demás. Un modelo puede representar adecuadamente colores o formas generales y, aun así, perder detalles necesarios para distinguir personajes parecidos. Por esta razón, la calidad de reconstrucción y la exactitud de clasificación se evaluaron por separado.

C. *Transferencia de aprendizaje con EfficientNetB0*

La transferencia de aprendizaje utiliza una red previamente entrenada como punto de partida para una nueva tarea. En lugar de aprender todas las características desde una inicialización aleatoria, se aprovecha una representación visual existente y se adapta la salida a las categorías de interés. EfficientNetB0 pertenece a una familia de redes convolucionales desarrollada mediante el estudio del escalado conjunto de profundidad, anchura y resolución (Tan & Le, 2019).

Se emplearon pesos preentrenados en ImageNet y una cabeza clasificadora específica para Pokémon. Primero se entrenó esa cabeza manteniendo fijo el extractor. Después se permitió modificar una parte de las capas finales con una tasa de aprendizaje menor. Esta segunda etapa, denominada ajuste fino, buscó adaptar las características a las imágenes del estudio sin alterar de manera brusca toda la representación preentrenada.

EfficientNetB0 también es una CNN. Por ello, la diferencia central entre las estrategias examinadas es cómo se obtiene y adapta la representación: supervisión desde cero, aprendizaje por reconstrucción o transferencia de conocimiento visual previo. Esta distinción orienta la interpretación de los resultados.

III. Metodología

La metodología comprendió seis componentes: preparación y auditoría del conjunto de datos, análisis exploratorio, preprocesamiento de imágenes, construcción de modelos, entrenamiento y selección mediante validación, y evaluación final. Las transformaciones de entrenamiento y las funciones de evaluación se implementaron en módulos separados para mantener un procedimiento consistente y facilitar su reproducción.

A. *Conjunto de datos*

Se utilizó el conjunto público `keremberke/pokemon-classification`, configuración `full`, disponible en Hugging Face. La publicación contiene 6,991 imágenes y enumera 150 categorías; sus particiones originales incluyen 4,869 imágenes de entrenamiento, 1,390 de validación y 732 de prueba (keremberke, s.f.). La partición empleada en el estudio se preparó mediante agrupación de imágenes relacionadas, en lugar de asumir que la distribución original garantizaba independencia entre subconjuntos.

Las etiquetas se representaron mediante índices enteros asociados con un orden fijo de categorías. Este mismo orden se utilizó durante entrenamiento, evaluación e inferencia, de manera que cada posición de la salida conservara una interpretación única. El número de clase se trató como identificador nominal, sin relación numérica con la identidad del personaje ni con su número de Pokédex.

B. *Auditoría y control de imágenes relacionadas*

Antes del entrenamiento se verificó la decodificación de los archivos y se registraron sus dimensiones, etiquetas y origen. La revisión contempló imágenes duplicadas, variantes de una misma ilustración y posibles contradicciones entre etiquetas. Estos controles son necesarios porque la presencia de contenido equivalente en entrenamiento y prueba puede sobreestimar la capacidad de generalización del modelo.

Se calcularon huellas SHA-256 del archivo y de su contenido visual decodificado. La primera permitió detectar modificaciones de los archivos y la segunda identificar imágenes con los mismos píxeles. Además, se empleó un hash perceptual de 64 bits para proponer pares visualmente similares. Este último se obtuvo al reducir una versión en escala de grises a una cuadrícula de 8×9 y comparar píxeles vecinos. Se revisaron como candidatos los pares con distancia de Hamming menor o igual que dos.

Los duplicados y las variantes confirmadas se agruparon para mantener una misma fuente visual dentro de un solo subconjunto. Los candidatos perceptuales requirieron revisión antes de considerarse equivalentes. Se excluyeron dos imágenes asociadas con una contradicción entre las etiquetas Poliwhirl y Poliwrath, conservando

los originales para mantener trazabilidad. La auditoría local registró cero fallos de decodificación y cero cruces de los grupos identificados entre particiones.

C. *División de entrenamiento, validación y prueba*

La distribución se realizó por clase y por grupo, con semilla 42. Se asignó aproximadamente 70 % de los grupos a entrenamiento, 15 % a validación y 15 % a prueba. La asignación por grupos y el redondeo por categoría produjeron los tamaños presentados en la Tabla I.

TABLA I
DISTRIBUCIÓN DE LAS IMÁGENES EN EL EXPERIMENTO FINAL

Subconjunto	Imágenes	Porcentaje	Imágenes por clase
Entrenamiento	4,898	70.08 %	25–46
Validación	1,045	14.95 %	5–10
Prueba	1,046	14.97 %	5–10
Total	6,989	100.00 %	150 categorías

El conjunto de entrenamiento se utilizó para actualizar parámetros. La validación permitió supervisar las pérdidas y seleccionar los pesos de cada etapa. La prueba se utilizó para evaluar el modelo final exportado sin actualización de parámetros. Esta separación evita utilizar el desempeño de prueba como criterio de selección dentro del procedimiento de entrenamiento.

La agrupación reduce la presencia de variantes conocidas entre subconjuntos; su alcance depende de las relaciones que logren identificarse. No constituye una garantía de que todas las imágenes procedan de fuentes independientes. La discrepancia de identificación entre copias del manifiesto se considera adicionalmente en las limitaciones de reproducibilidad.

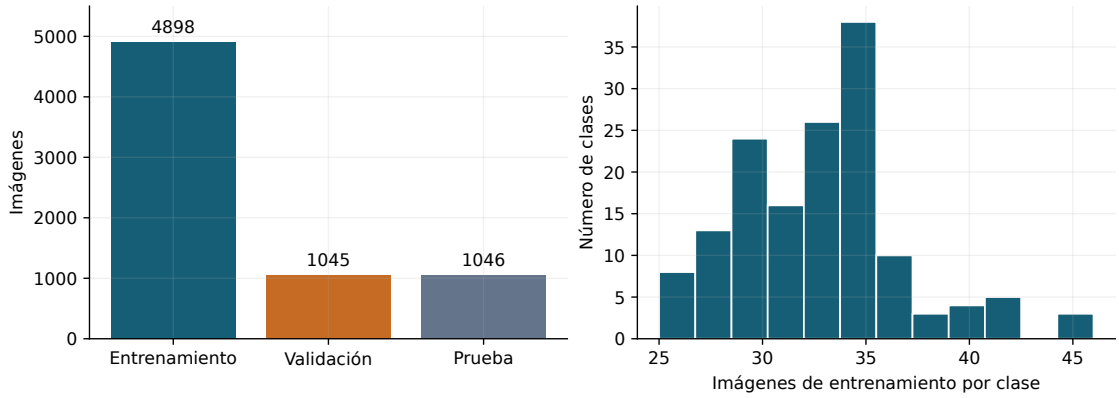


Fig. 1. Distribución de imágenes por subconjunto y frecuencia de muestras de entrenamiento por categoría.

D. *Análisis exploratorio*

El análisis exploratorio examinó el número de categorías, su frecuencia y las dimensiones de las imágenes de entrenamiento. Las 150 categorías estuvieron representadas en los tres subconjuntos. En entrenamiento, el soporte varió entre 25 y 46 imágenes por clase; en validación y prueba, entre cinco y diez. Esta distribución permitió evaluar todas las clases, aunque el soporte reducido de evaluación exige cautela al interpretar métricas individuales.

Las imágenes de entrenamiento presentaron dimensiones de 224×224 píxeles, relación de aspecto igual a uno y desviación nula en ancho y alto. La inspección de muestras permitió examinar diferencias de pose, fondo y estilo dentro del conjunto. Estos aspectos son relevantes porque el modelo debe reconocer la categoría sin

depender exclusivamente de una composición o un fondo particular.

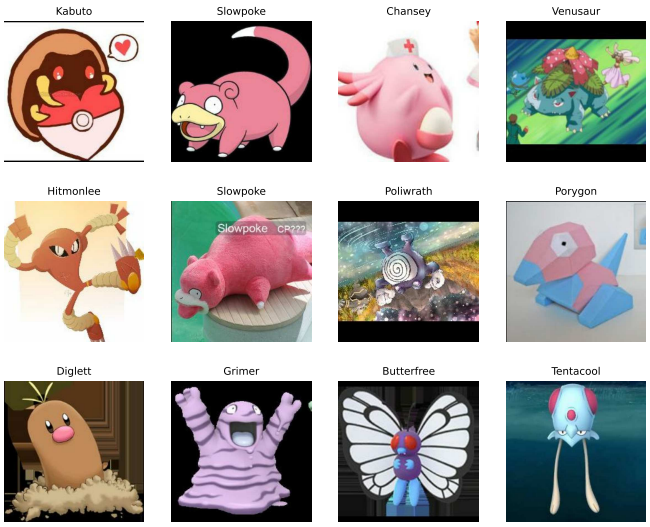


Fig. 2. Muestra de imágenes del subconjunto de entrenamiento.

E. Preprocesamiento y aumento de datos

Las imágenes JPEG y PNG se decodificaron a tres canales RGB. Las transparencias se compusieron sobre un fondo blanco. Después se ajustó el tamaño a 224×224 píxeles conservando las proporciones y completando las áreas restantes con relleno blanco. El procedimiento entregó tensores de tipo float32 en el intervalo $[0, 255]$.

La normalización se aplicó de acuerdo con cada modelo. La CNN y el encoder del autoencoder incluyeron una división por 255 para trabajar en $[0, 1]$. EfficientNetB0 recibió directamente valores en $[0, 255]$, conforme al preprocesamiento incorporado en la implementación de Keras (Keras, s.f.-a). Mantener este criterio también durante inferencia evitó diferencias entre las imágenes utilizadas para aprender y las utilizadas para predecir.

El aumento de datos se aplicó exclusivamente en entrenamiento mediante volteo horizontal y rotaciones aleatorias de aproximadamente $\pm 14{,}4^\circ$, con relleno blanco. Estas transformaciones introdujeron variaciones de orientación sin cambiar la etiqueta. Validación y prueba se procesaron de forma determinista, sin aumentos.

Para el autoencoder, la transformación se aplicó antes de construir el par de aprendizaje. La entrada fue la imagen transformada y el objetivo fue esa misma imagen normalizada. Así, la reconstrucción mantuvo correspondencia espacial con la entrada. El procedimiento no incorporó una tarea adicional de eliminación de ruido.

IV. Modelos y configuración experimental

A. CNN supervisada desde cero

La CNN se construyó con tres bloques convolucionales de 32, 64 y 128 filtros. Cada bloque incluyó dos convoluciones 3×3 con activación ReLU, conservación de dimensiones y una operación MaxPooling de factor dos. Esta organización permitió aumentar progresivamente la cantidad de características mientras se reducía la resolución espacial.

Después de los bloques se aplicó un promedio global de los mapas de activación. La representación resultante se conectó a una capa densa de 128 unidades y a dropout de 0.3. Finalmente, una capa de 150 unidades con softmax produjo la distribución de categorías. El dropout introdujo regularización durante entrenamiento para reducir la dependencia de combinaciones particulares de activaciones.

TABLA II
ARQUITECTURA DE LA CNN CON INICIALIZACIÓN ALEATORIA

Etapa	Operación	Dimensiones de salida
Entrada	RGB y normalización	$224 \times 224 \times 3$
Bloque 1	Dos Conv(32) y MaxPool(2)	$112 \times 112 \times 32$
Bloque 2	Dos Conv(64) y MaxPool(2)	$56 \times 56 \times 64$
Bloque 3	Dos Conv(128) y MaxPool(2)	$28 \times 28 \times 128$
Representación	Promedio global	128
Clasificador	Dense(128), Dropout(0.3), Dense(150)	150

La arquitectura comprendió 322,870 parámetros. Se configuró Adam con tasa inicial 10^{-3}, lotes de 32 imágenes, un máximo de 40 épocas y detención temprana con paciencia de seis. El análisis experimental de esta red quedó restringido a dos épocas registradas; no se dispuso de una evaluación final para establecer su rendimiento tras un entrenamiento completo.

B. Autoencoder convolucional y clasificador

El encoder se implementó mediante tres convoluciones con paso de dos y 32, 64 y 128 filtros. Esta reducción progresiva transformó la imagen en mapas de $28 \times 28 \times 128$. A continuación, el promedio global y una capa densa produjeron un vector latente de 128 componentes. Este vector representó el punto de compresión utilizado tanto para reconstruir como para clasificar.

El decoder expandió el vector mediante una capa densa y lo reorganizó en un volumen de $28 \times 28 \times 64$. Tres etapas de aumento de resolución, seguidas de convoluciones con 64, 32 y 16 filtros, recuperaron el tamaño espacial de la entrada. Una convolución final de tres canales y activación sigmoid produjo la reconstrucción RGB en $[0, 1]$. La arquitectura no incluyó conexiones directas entre las capas del encoder y del decoder.

El entrenamiento se dividió en dos fases. Primero se ajustaron encoder y decoder para minimizar el error de reconstrucción. Después se congelaron los parámetros del encoder y se entrenó una cabeza con Dense(128), Dropout(0.3) y Dense(150, softmax). La segunda fase permitió evaluar la información discriminativa del vector latente sin modificar la representación aprendida mediante reconstrucción.

El autoencoder completo tuvo 6,642,531 parámetros: 109,760 en el encoder y 6,532,771 en el decoder. El clasificador utilizó únicamente el encoder y su nueva cabeza, por lo que el decoder no intervino en la predicción de categorías. Se configuraron lotes de 32 imágenes, tasa inicial 10^{-3} y un máximo de 40 épocas por fase. No se aplicó ajuste fino supervisado al encoder después de entrenar la cabeza.

C. EfficientNetB0 con transferencia de aprendizaje

Se utilizó EfficientNetB0 preentrenada en ImageNet y sin su clasificador original. El promedio global del extractor produjo un vector de 1,280 características. Sobre él se incorporaron una capa densa de 128 unidades con ReLU, dropout de 0.3 y una salida softmax de 150 categorías. La red completa comprendió 4,232,889 parámetros, de los cuales 183,318 correspondieron a la cabeza nueva.

El experimento final se desarrolló en dos etapas. Durante la primera se mantuvo congelado el extractor y se entrenó exclusivamente la cabeza con tasa inicial 10^{-3}. En la segunda se habilitó el ajuste de las capas finales correspondientes a `block7` y `top_`, manteniendo congeladas las capas de normalización por lotes. La tasa inicial se redujo a 10^{-5} y el modelo se recompiló con un nuevo optimizador.

Mantener las capas de normalización en modo inferencia permitió conservar sus estadísticas durante la adaptación. El uso de una tasa menor en el ajuste fino buscó realizar modificaciones graduales sobre las características preentrenadas, de acuerdo con el procedimiento de transferencia de aprendizaje de Keras (Keras, s.f.-b). El número de parámetros entrenables en esta etapa fue 1,304,902, incluida la cabeza.

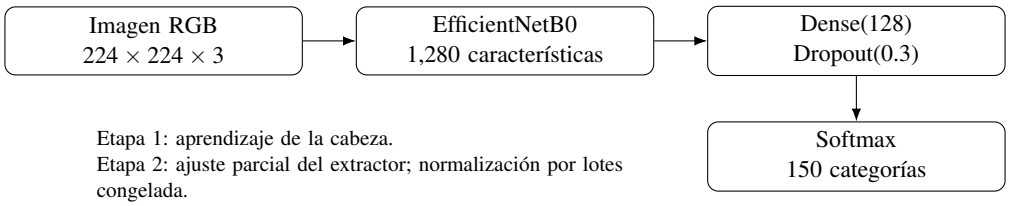


Fig. 3. Arquitectura del clasificador basado en EfficientNetB0.

También se evaluó una configuración de EfficientNetB0 en Colab con 40 épocas de entrenamiento de cabeza. Sus resultados se presentan por separado de los del experimento final, realizado con lotes de ocho imágenes y ajuste fino parcial. La diferencia entre estas configuraciones se consideró al interpretar sus métricas, sin atribuir el cambio de desempeño a una única modificación.

D. Funciones de pérdida y optimización

Los clasificadores se entrenaron con entropía cruzada categórica. Esta función penaliza que el modelo asigne una probabilidad baja a la categoría correcta:

$$\mathscr{L}_{\text{cls}} = -\frac{1}{N}\sum_{i=1}^{N} \log p(y_i \mid x_i). \tag{1}$$

En esta expresión, N es el número de imágenes del lote y $p(y_i \mid x_i)$ es la probabilidad asignada a la etiqueta correcta de la imagen i. La función utiliza directamente las etiquetas enteras y permite actualizar los parámetros para favorecer la identificación de la clase de referencia.

La reconstrucción del autoencoder se optimizó con error cuadrático medio:

$$\mathscr{L}_{\text{rec}} = \frac{1}{NHWC}\sum_{i=1}^{N}\sum_{h,w,c}\left(x'_{ihwc} - \hat{x}_{ihwc}\right)^2. \tag{2}$$

Aquí, x' es la imagen normalizada a $[0,1]$ y $\hat{x}$ su reconstrucción; H, W y C representan alto, ancho y canales. La función resume las diferencias de intensidad por píxel, por lo que su escala no es directamente comparable con la pérdida de clasificación.

En todas las estrategias se utilizó Adam, que adapta las actualizaciones a partir de estimaciones de los momentos del gradiente (Kingma & Ba, 2015). La tasa de aprendizaje inicial y el número de etapas se ajustaron a la configuración de cada modelo.

E. Selección de pesos y control del entrenamiento

Se conservaron el mejor modelo según pérdida de validación y el último estado de cada época. La detención temprana se configuró con paciencia de seis épocas y restauración de los mejores pesos. Además, la tasa de aprendizaje se redujo a la mitad tras dos épocas sin mejora, con un mínimo de 10^{-7}. La ejecución se interrumpió ante pérdidas no finitas.

En EfficientNetB0 se comparó la menor pérdida de validación de la cabeza con la del ajuste fino. Se eligió la segunda etapa únicamente si reducía esa pérdida. Este criterio conservó una regla uniforme de selección: el modelo final correspondió al menor error de validación, aunque otra época pudiera presentar una exactitud ligeramente mayor.

La implementación se organizó en módulos de datos, arquitecturas, entrenamiento, métricas y evaluación. Se fijó la semilla 42 y se conservaron configuraciones, versiones, identificadores de datos, modelos y predicciones. El experimento final se ejecutó con Python 3.12.3 y TensorFlow 2.20.0 en WSL2 con GPU. El autoencoder y la configuración inicial de EfficientNetB0 se ejecutaron en Colab con Tesla T4; la CNN se ejecutó en CPU bajo macOS. Las mediciones de duración corresponden al entorno de cada experimento.

V. Evaluación

A. Métricas globales y por categoría

Se evaluaron exactitud top-1, exactitud top-3, precisión macro, recall macro y F1 macro. La exactitud top-1 expresa la proporción de imágenes cuya primera predicción coincide con la etiqueta de referencia. La exactitud top-3 indica en cuántas imágenes la categoría correcta aparece entre las tres candidatas principales. Esta segunda medida permite estudiar la utilidad de presentar alternativas, sin equipararla con una decisión automática única.

Para una categoría, la precisión mide qué proporción de las imágenes asignadas a ella realmente le pertenecen. El recall mide qué proporción de sus imágenes fue identificada correctamente. El F1 resume el equilibrio entre ambas medidas:

$$P_c = \frac{TP_c}{TP_c + FP_c}, \qquad R_c = \frac{TP_c}{TP_c + FN_c}, \qquad F1_c = \frac{2P_cR_c}{P_c + R_c}. \tag{3}$$

Los términos TP, FP y FN representan verdaderos positivos, falsos positivos y falsos negativos de la categoría c. El promedio macro se calculó como la media de cada métrica sobre las 150 categorías; las divisiones por cero se resolvieron con cero. De esta manera, cada categoría tuvo el mismo peso en la evaluación macro, independientemente de su frecuencia.

B. Evaluación final y análisis de errores

El modelo seleccionado se exportó junto con su mapa de etiquetas y se volvió a cargar para la evaluación de prueba. Se verificó la correspondencia del orden de clases y se mantuvo el mismo preprocesamiento. La evaluación se ejecutó en modo inferencia, sin actualizar parámetros.

Se conservaron la matriz de confusión, las métricas por clase y las probabilidades de las 150 categorías para cada imagen. La matriz ubicó las etiquetas reales en filas y las predicciones en columnas. El análisis se centró en las categorías de menor F1 y en las confusiones direccionales más frecuentes, con el fin de interpretar qué dificultades persistían después del entrenamiento.

Los resultados de validación de configuraciones distintas se presentaron de forma descriptiva. No se realizó una prueba estadística de superioridad entre arquitecturas, debido a la ausencia de repeticiones y a las diferencias de configuración. La evaluación en prueba se reservó, en este artículo, al clasificador final de EfficientNetB0.

VI. Resultados

A. Comportamiento inicial de la CNN

En las dos épocas registradas de la CNN, la exactitud de entrenamiento pasó de 0.67 % a 1.82 %, mientras que la de validación pasó de 0.86 % a 0.67 %. La pérdida de validación aumentó de 5.01096 a 6.22444. Estos valores corresponden al inicio del ajuste y no permiten caracterizar el rendimiento que la arquitectura alcanzaría tras completar su entrenamiento.

Por tanto, la CNN no se incluyó en la comparación de métricas finales. Su implementación permitió establecer una referencia de aprendizaje supervisado desde cero, pero la evaluación cuantitativa de esa estrategia permanece incompleta.

B. Reconstrucción y clasificación mediante autoencoder

El autoencoder ejecutó 21 épocas de reconstrucción y 40 épocas de entrenamiento de cabeza. La pérdida MSE de validación disminuyó de aproximadamente 0.0910 a un mínimo de 0.0610. La evolución de ambas pérdidas se presenta en la Figura 4, junto con la exactitud del clasificador.

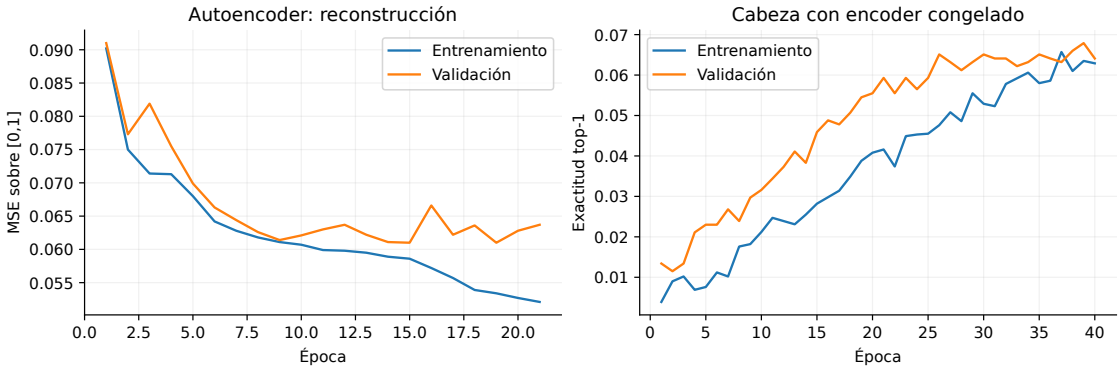


Fig. 4. Entrenamiento del autoencoder: pérdida de reconstrucción y exactitud de clasificación con el encoder congelado.

La clasificación alcanzó una exactitud de validación de 6.79 %, top-3 de 16.65 %, precisión macro de 4.32 %, recall macro de 6.23 % y F1 macro de 4.15 %, sobre 1,045 imágenes. La mejora de reconstrucción no se acompañó de una capacidad elevada para distinguir categorías. El resultado indica que la representación latente, tal como fue aprendida y utilizada, no conservó información discriminativa suficiente para resolver la tarea con la cabeza implementada.

C. Resultados de validación por configuración

La configuración inicial de EfficientNetB0 obtuvo una exactitud de validación de 75.22 % y un F1 macro de 74.68 % después de 40 épocas de entrenamiento de cabeza. En el experimento final, la exactitud fue 87.08 % y el F1 macro 86.95 %. La Tabla III reúne estas mediciones y las del autoencoder.

TABLA III
MÉTRICAS DE VALIDACIÓN DE LAS CONFIGURACIONES EVALUADAS

Configuración	Top-1	Top-3	P macro	R macro	F1 macro
Autoencoder y clasificador	6.79 %	16.65 %	4.32 %	6.23 %	4.15 %
EfficientNetB0 inicial	75.22 %	90.05 %	82.23 %	74.93 %	74.68 %
EfficientNetB0 final	87.08 %	95.12 %	88.53 %	87.09 %	86.95 %

Estas cifras describen el desempeño observado de cada configuración. Las diferencias de lote, entorno y etapas de entrenamiento impiden atribuir el cambio entre las dos ejecuciones de EfficientNetB0 exclusivamente al ajuste fino. El efecto de esta etapa se examinó dentro del experimento final, manteniendo su procedimiento de entrenamiento y validación.

D. Entrenamiento y ajuste fino de EfficientNetB0

La etapa de cabeza ejecutó 34 épocas y alcanzó su menor pérdida de validación en la época 28. El ajuste fino ejecutó 17 épocas y alcanzó su menor pérdida en la época 11. Los valores de época se expresan desde uno y se refieren a cada etapa de manera independiente.

TABLA IV
RESULTADOS DE ENTRENAMIENTO DEL EXPERIMENTO FINAL

Propiedad	Cabeza	Ajuste fino
Tasa de aprendizaje inicial	10^{-3}	10^{-5}
Tamaño del lote	8	8
Máximo de épocas	40	20
Épocas ejecutadas	34	17
Época seleccionada	28	11
Pérdida de validación seleccionada	0.50603	0.48299
Exactitud de validación seleccionada	85.93 %	87.08 %
Tiempo de entrenamiento (s)	1,761.98	962.64
Parámetros entrenables	183,318	1,304,902

El ajuste fino redujo la pérdida de validación aproximadamente 4.55 % respecto al checkpoint de cabeza y aumentó la exactitud en 1.15 puntos porcentuales. Por ello se seleccionó el modelo de la segunda etapa. El criterio de menor pérdida fue relevante: la última época presentó una exactitud de validación de 87.37 %, pero no correspondió al estado de menor pérdida y no se utilizó para la evaluación final.

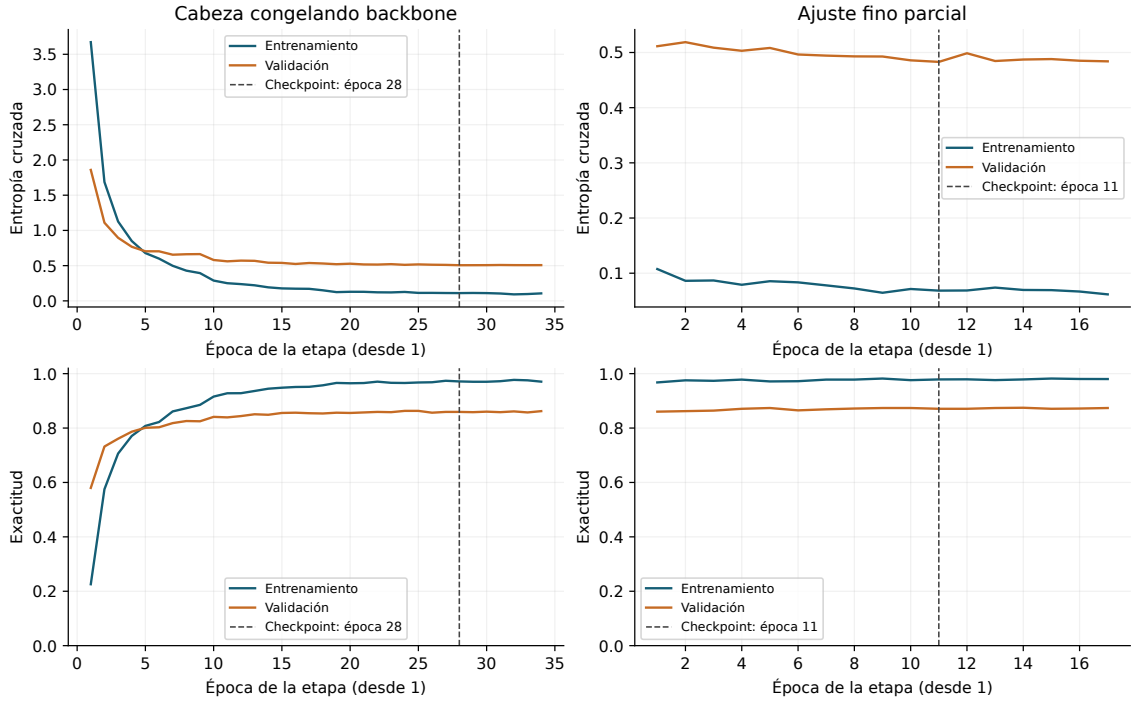


Fig. 5. Pérdida y exactitud durante el entrenamiento de cabeza y el ajuste fino parcial. Las líneas verticales señalan el modelo seleccionado por pérdida de validación.

El modelo seleccionado alcanzó una exactitud de entrenamiento de aproximadamente 97.88 %, frente a 87.08 % en validación. La diferencia muestra una brecha de generalización, pese a las medidas de regularización y selección. El tiempo total de ajuste fue 2,724.62 segundos, aproximadamente 45.41 minutos, sin incluir preparación de datos, evaluación, exportación ni el preentrenamiento en ImageNet. El archivo del clasificador ocupó aproximadamente 26.93 MiB.

E. Desempeño en el conjunto de prueba

La Tabla V presenta las métricas del modelo final. La exactitud top-1 fue 87.57 %, top-3 95.12 % y F1 macro 87.24 %. El resultado correspondió a 916 predicciones correctas y 130 errores entre 1,046 imágenes.

TABLA V
DESEMPEÑO DEL CLASIFICADOR FINAL DE EFFICIENTNETB0

Métrica	Validación ($n = 1,045$)	Prueba ($n = 1,046$)
Exactitud top-1	87.08 %	87.57 %
Exactitud top-3	95.12 %	95.12 %
Precisión macro	88.53 %	88.94 %
Recall macro	87.09 %	87.58 %
F1 macro	86.95 %	87.24 %

La categoría correcta apareció entre las tres principales en 995 imágenes. En consecuencia, 79 de los 130 errores de primera predicción conservaron la respuesta correcta dentro de las alternativas top-3; 51 imágenes quedaron fuera de ese conjunto. Esta diferencia muestra que parte de las dificultades se relacionó con el orden entre candidatas, aunque la decisión principal continuó siendo incorrecta en esos casos.

La exactitud de prueba superó a la de validación en 0.49 puntos porcentuales y el F1 macro en 0.29 puntos. La proximidad de los resultados sugiere un comportamiento semejante en ambas particiones, sin que ello sustituya una evaluación en datos de una fuente externa.

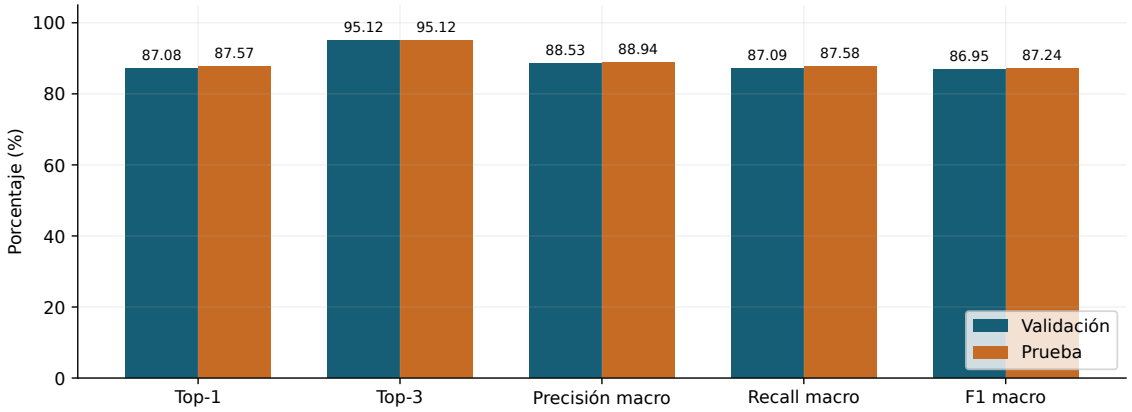


Fig. 6. Métricas de validación y prueba del clasificador final.

F. Análisis de errores por categoría

El desempeño no fue uniforme entre categorías. Treinta y dos clases alcanzaron F1 igual a uno en la muestra de prueba; las de menor F1 se presentan en la Tabla VI. Estos valores deben interpretarse junto con el soporte, ya que cada categoría incluyó entre cinco y diez imágenes.

TABLA VI
CATEGORÍAS CON MENOR F1 EN PRUEBA

Categoría	Soporte	Precisión	Recall	F1
Pidgeotto	6	1.000	0.167	0.286
Kadabra	7	0.571	0.571	0.571
Alolan Sandslash	6	0.750	0.500	0.600
Zubat	6	0.750	0.500	0.600
Golduck	7	0.667	0.571	0.615
Cloyster	7	0.625	0.714	0.667
Fearow	7	0.800	0.571	0.667
Gyarados	7	0.545	0.857	0.667

Pidgeotto presentó el menor F1, con un valor de 0.286 y recall de 0.167. Tres de sus imágenes se clasificaron como Pidgeot, lo que constituyó la confusión direccional más frecuente. También se observaron dos errores en cada una de las siguientes direcciones: Alakazam hacia Kadabra, Doduo hacia Dodrio, Machamp hacia Rhydon y Muk hacia Grimer.

Las confusiones entre personajes de una misma línea evolutiva son compatibles con la presencia de rasgos compartidos. Sin embargo, la aparición de errores como Machamp hacia Rhydon muestra que las dificultades no se limitaron a esas relaciones. La matriz de confusión identifica los pares afectados, pero no permite determinar por sí sola qué región o característica visual originó cada decisión.

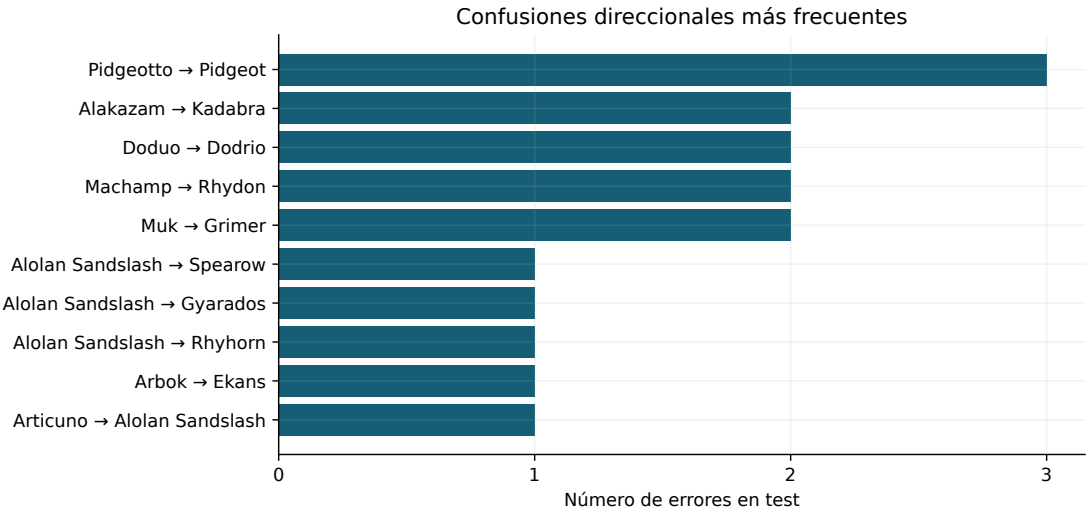


Fig. 7. Confusiones direccionales más frecuentes del modelo final en prueba.

Las métricas completas de las 150 categorías se presentan en el apéndice. La matriz de confusión completa se conserva como material suplementario, junto con las predicciones individuales. Esto permite examinar el resultado tanto a nivel global como por clase.

VII. IMPLEMENTACIÓN DE INFERENCIA

El clasificador se exportó con su arquitectura, pesos, mapa de etiquetas y especificaciones de entrada. La consistencia entre la versión entrenada y la recargada se comprobó mediante tolerancias numéricas de 10^{-5} en términos absolutos y 10^{-4} en términos relativos. La identidad del modelo se verificó mediante SHA-256 para mantener correspondencia entre evaluación e inferencia.

Se desarrolló una interfaz local denominada Pokémon Center para cargar imágenes y mostrar las cinco categorías con mayor valor softmax. El servidor cargó el modelo al iniciar y utilizó el mismo preprocesamiento de entrenamiento. La interfaz permitió presentar los resultados de forma accesible sin modificar los parámetros del clasificador.

Una imagen de Alakazam produjo esta categoría como primera candidata, con un valor softmax de 99.71 %. En otro ejemplo, una imagen identificada por su nombre como Zapdos produjo Vaporeon como primera candidata, con 65.65 %, y Zapdos como segunda, con 18.29 %. Los ejemplos se utilizaron únicamente para comprobar el funcionamiento y visualizar decisiones; no se incorporaron al cálculo de las métricas de prueba, y su identificación por nombre de archivo no sustituyó un etiquetado independiente.

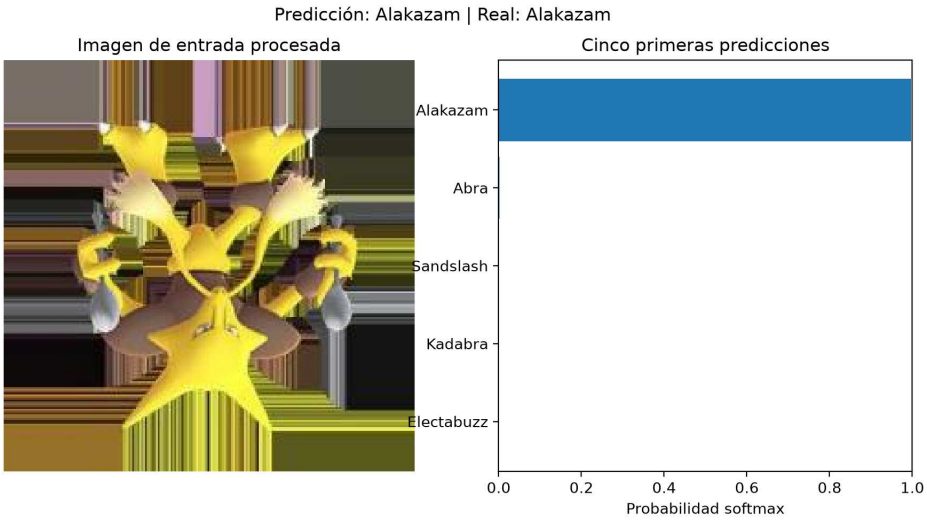


Fig. 8. Ejemplo de inferencia individual con Alakazam como primera candidata.

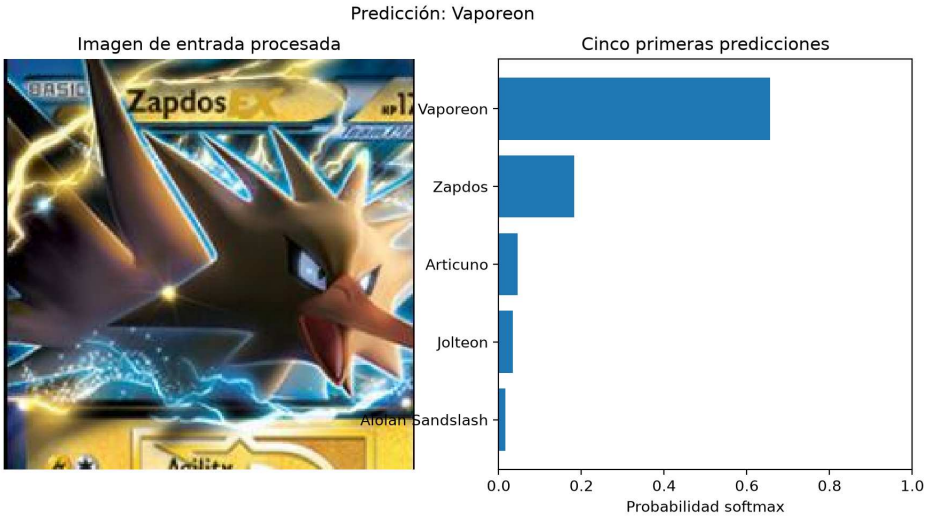


Fig. 9. Ejemplo de inferencia en el archivo Zapdos.jpg: Vaporeon ocupó la primera posición y Zapdos la segunda.

La distribución softmax expresa la preferencia del clasificador entre las categorías disponibles. Un valor elevado no constituye una certeza calibrada ni garantiza que la imagen pertenezca a una clase conocida. Por ello, la visualización de candidatos se interpretó como resultado de inferencia, manteniendo separada su presentación

de la evaluación cuantitativa.

VIII. Discusión

A. *Capacidad discriminativa de las representaciones*

El clasificador final alcanzó un F1 macro de 87.24 % en prueba, lo que indica un desempeño elevado dentro de las condiciones evaluadas. La cercanía entre F1 macro y exactitud sugiere que el resultado global no dependió exclusivamente de las categorías con mayor número de muestras. Sin embargo, las diferencias entre clases muestran que persistieron problemas específicos de reconocimiento.

El autoencoder obtuvo una reducción de la pérdida de reconstrucción, pero su clasificador mantuvo un rendimiento limitado. Esta diferencia resulta coherente con el objetivo de aprendizaje: reconstruir una imagen favorece conservar información visual, mientras que clasificar requiere que esa información permita separar categorías. En la configuración examinada, la representación de 128 componentes y el encoder congelado no fueron suficientes para lograr esa separación con la cabeza utilizada.

La compresión espacial y el entrenamiento orientado a píxeles podrían contribuir a la pérdida de detalles discriminativos. No obstante, identificar su efecto requeriría comparar variantes de la dimensión latente, del objetivo de aprendizaje o del ajuste supervisado del encoder. Los resultados permiten describir una limitación de la configuración implementada, sin extenderla a todos los autoencoders.

B. *Contribución del ajuste fino*

La comparación interna del experimento final mostró una reducción de la pérdida de validación y una mejora de 1.15 puntos porcentuales en exactitud después del ajuste fino. Esta observación respalda la utilidad de adaptar parcialmente la representación preentrenada, además de aprender una nueva cabeza. El uso de una tasa menor y el ajuste de un subconjunto de capas permitieron realizar esa adaptación de forma limitada.

El efecto observado debe distinguirse de la diferencia entre las dos configuraciones completas de EfficientNetB0. Sus resultados proceden de condiciones distintas y no permiten aislar un único factor. Asimismo, la transferencia incorpora conocimiento aprendido previamente en ImageNet, por lo que una comparación con inicialización aleatoria no puede atribuirse únicamente a la estructura de la red.

La brecha entre exactitud de entrenamiento y validación indica que parte de la información aprendida no se trasladó con la misma eficacia a ejemplos distintos. La regularización, los aumentos y la selección por validación redujeron el riesgo de conservar un estado sobreajustado, pero no eliminaron esa diferencia. Esto justifica estudiar más datos y variaciones de regularización en futuras evaluaciones.

C. *Interpretación de los errores y utilidad del top-3*

El bajo recall de Pidgeotto y su confusión con Pidgeot muestran una dificultad para separar categorías con características cercanas. En este contexto, la exactitud global resume el funcionamiento del sistema, pero no describe por completo su comportamiento. Las métricas por clase permiten localizar estas debilidades y orientar nuevas hipótesis sin modificar retrospectivamente la evaluación final.

La exactitud top-3 de 95.12 % indica que una lista corta de alternativas puede ofrecer información útil cuando la primera predicción falla. En 79 casos, la categoría de referencia permaneció entre esas opciones. Esta propiedad podría aprovecharse en una herramienta de apoyo con revisión humana; no representa el mismo porcentaje de decisiones automáticas correctas.

La interfaz permitió comprobar la integración del modelo y presentar las alternativas. Su utilidad funcional es complementaria a la evaluación: una predicción individual, incluso con valor softmax alto, no reemplaza el desempeño medido sobre una muestra etiquetada.

IX. Limitaciones

La evaluación se realizó con una única semilla y un conjunto de prueba de 1,046 imágenes. No se estimaron intervalos de confianza ni variabilidad entre repeticiones. El soporte de cinco a diez imágenes por clase hace que sus métricas sean sensibles a pocos errores; por tanto, un F1 perfecto en una categoría no implica reconocimiento perfecto de futuras imágenes.

La comparación entre estrategias está limitada por la evaluación incompleta de la CNN y por diferencias de configuración entre ejecuciones. Además, los experimentos auxiliares no cuentan con una verificación completa de equivalencia entre sus particiones y las del experimento final. Por estas razones, sus métricas de validación se interpretan de manera descriptiva y no como una demostración controlada de superioridad entre arquitecturas.

La identificación del manifiesto de datos presenta una discrepancia entre una copia local y la huella asociada al entrenamiento final. Esto limita la reproducción exacta a partir de esa copia, aunque las métricas de prueba se encuentran respaldadas por las predicciones y la matriz de confusión de la evaluación. La resolución de esa discrepancia es necesaria para completar la trazabilidad del conjunto utilizado en el ajuste.

La generalización se evaluó dentro de la fuente de datos seleccionada. No se realizó validación externa sobre fotografías de objetos, dibujos a mano, escenas con varios personajes o imágenes con oclusiones. Tampoco se evaluó rechazo de categorías desconocidas, calibración de probabilidades o explicabilidad mediante mapas de activación. Estas condiciones delimitan la aplicación de las conclusiones al dominio estudiado.

Finalmente, el tiempo de entrenamiento informado no incluye el costo previo de ImageNet ni constituye una comparación de eficiencia entre todos los modelos. No se midieron latencia de inferencia, consumo energético ni memoria máxima, por lo que no se formulan conclusiones sobre esas dimensiones.

X. Conclusiones

Se implementó un procedimiento de clasificación visual de Pokémon que integró auditoría de datos, agrupación de imágenes relacionadas, análisis exploratorio, preprocesamiento común, aprendizaje de representaciones, selección mediante validación y evaluación del modelo exportado. La preparación del conjunto y la conservación del mapa de categorías permitieron mantener consistencia entre entrenamiento e inferencia.

EfficientNetB0 con transferencia de aprendizaje y ajuste fino parcial obtuvo una exactitud de 87.57 %, exactitud top-3 de 95.12 % y F1 macro de 87.24 % en prueba. El ajuste fino mejoró la exactitud de validación aproximadamente 1.15 puntos porcentuales respecto a la cabeza entrenada con el extractor congelado. Estos resultados respaldan su uso como clasificador experimental para las 150 categorías consideradas.

El autoencoder produjo una representación capaz de reducir el error de reconstrucción, pero su clasificación alcanzó un F1 macro de 4.15 % en validación. El resultado muestra que la conservación de información visual mediante reconstrucción no fue suficiente para obtener una representación discriminativa en la configuración evaluada. La CNN desde cero requiere completar su entrenamiento y evaluación antes de incorporarse a una comparación cuantitativa definitiva.

Las siguientes etapas de investigación deben priorizar la verificación completa de las particiones, la evaluación de todas las estrategias bajo condiciones comunes, las repeticiones con distintas semillas y la validación externa. El ajuste supervisado del encoder, la calibración de probabilidades y el análisis de regiones relevantes constituyen extensiones que permitirían profundizar en las causas de los errores y en la utilidad del sistema.

REFERENCIAS

Hinton, G. E., & Salakhutdinov, R. R. (2006). Reducing the Dimensionality of Data with Neural Networks. *Science*, *313*(5786), 504-507. https://doi.org/10.1126/science.1127647

Keras. (s.f.-a). *EfficientNet models: EfficientNetB0*. Consultado el 3 de noviembre de 2026, desde https://keras.io/api/applications/efficientnet/efficientnet_models/

Keras. (s.f.-b). *Transfer learning and fine-tuning*. Consultado el 3 de noviembre de 2026, desde https://keras.io/guides/transfer_learning/

keremberke. (s.f.). *pokemon-classification: Dataset card*. Consultado el 3 de noviembre de 2026, desde https://huggingface.co/datasets/keremberke/pokemon-classification

Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *International Conference on Learning Representations*. https://arxiv.org/abs/1412.6980

Tan, M., & Le, Q. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *Proceedings of the 36th International Conference on Machine Learning*, *97*, 6105-6114. https://proceedings.mlr.press/v97/tan19a.html

A. DESEMPEÑO POR CATEGORÍA EN PRUEBA

La tabla siguiente presenta precisión, recall y F1 en proporciones entre cero y uno. El soporte corresponde al número de imágenes de cada categoría en el conjunto de prueba. El orden de las filas coincide con el mapa de etiquetas del clasificador.

TABLA VII
MÉTRICAS POR CATEGORÍA DEL MODELO FINAL

Categoría	Soporte	Precisión	Recall	F1
Abra	5	0.714	1.000	0.833
Aerodactyl	5	0.625	1.000	0.769
Alakazam	9	1.000	0.778	0.875
Alolan Sandslash	6	0.750	0.500	0.600
Arbok	6	1.000	0.833	0.909
Arcanine	6	0.600	1.000	0.750
Articuno	7	0.750	0.857	0.800
Beedrill	7	1.000	1.000	1.000
Bellsprout	6	1.000	1.000	1.000
Blastoise	6	1.000	0.833	0.909
Bulbasaur	6	1.000	1.000	1.000
Butterfree	6	0.857	1.000	0.923
Caterpie	6	1.000	1.000	1.000
Chansey	6	1.000	0.833	0.909
Charizard	7	0.833	0.714	0.769
Charmander	8	1.000	1.000	1.000
Charmeleon	7	1.000	0.714	0.833
Clefable	6	0.833	0.833	0.833
Clefairy	7	0.857	0.857	0.857
Cloyster	7	0.625	0.714	0.667
Cubone	6	1.000	1.000	1.000
Dewgong	6	1.000	0.833	0.909
Diglett	6	1.000	0.833	0.909
Ditto	6	0.857	1.000	0.923
Dodrio	6	0.667	1.000	0.800
Doduo	5	1.000	0.600	0.750
Dragonair	6	0.750	1.000	0.857

Categoría	Soporte	Precisión	Recall	F1
Dragonite	7	0.875	1.000	0.933
Dratini	9	1.000	0.778	0.875
Drowzee	6	0.750	1.000	0.857
Dugtrio	8	0.875	0.875	0.875
Eevee	6	0.857	1.000	0.923
Ekans	7	0.857	0.857	0.857
Electabuzz	6	1.000	0.833	0.909
Electrode	8	0.889	1.000	0.941
Exeggcute	7	1.000	1.000	1.000
Exeggutor	8	0.875	0.875	0.875
Farfetchd	7	1.000	0.857	0.923
Fearow	7	0.800	0.571	0.667
Flareon	7	1.000	0.857	0.923
Gastly	7	1.000	1.000	1.000
Gengar	7	1.000	1.000	1.000
Geodude	5	0.833	1.000	0.909
Gloom	7	0.875	1.000	0.933
Golbat	7	0.700	1.000	0.824
Goldeen	7	0.714	0.714	0.714
Golduck	7	0.667	0.571	0.615
Golem	7	0.875	1.000	0.933
Graveler	6	0.857	1.000	0.923
Grimer	7	0.750	0.857	0.800
Growlithe	7	1.000	0.857	0.923
Gyarados	7	0.545	0.857	0.667
Haunter	7	0.857	0.857	0.857
Hitmonchan	7	0.857	0.857	0.857
Hitmonlee	7	1.000	0.857	0.923
Horsea	8	1.000	1.000	1.000
Hypno	7	0.833	0.714	0.769
Ivysaur	6	1.000	0.833	0.909
Jigglypuff	7	0.875	1.000	0.933
Jolteon	8	0.889	1.000	0.941
Jynx	7	1.000	1.000	1.000
Kabuto	7	1.000	1.000	1.000
Kabutops	6	1.000	1.000	1.000
Kadabra	7	0.571	0.571	0.571
Kakuna	7	1.000	1.000	1.000
Kangaskhan	8	1.000	0.625	0.769
Kingler	7	0.667	0.857	0.750
Koffing	7	1.000	1.000	1.000
Krabby	7	0.800	0.571	0.667
Lapras	8	0.875	0.875	0.875
Lickitung	7	0.875	1.000	0.933
Machamp	8	1.000	0.750	0.857
Machoke	6	0.833	0.833	0.833
Machop	6	1.000	0.833	0.909
Magikarp	7	1.000	1.000	1.000
Magmar	7	0.857	0.857	0.857
Magnemite	7	1.000	0.714	0.833
Magneton	6	1.000	1.000	1.000
Mankey	8	0.875	0.875	0.875
Marowak	10	1.000	1.000	1.000

Categoría	Soporte	Precisión	Recall	F1
Meowth	6	1.000	1.000	1.000
Metapod	8	1.000	0.750	0.857
Mew	8	0.857	0.750	0.800
Mewtwo	8	1.000	0.750	0.857
Moltres	6	0.750	1.000	0.857
MrMime	7	1.000	0.714	0.833
Muk	6	0.800	0.667	0.727
Nidoking	8	1.000	1.000	1.000
Nidoqueen	8	0.750	0.750	0.750
Nidorina	6	0.800	0.667	0.727
Nidorino	8	0.727	1.000	0.842
Ninetales	7	0.857	0.857	0.857
Oddish	8	1.000	0.875	0.933
Omanyte	7	0.857	0.857	0.857
Omastar	6	0.714	0.833	0.769
Onix	6	1.000	1.000	1.000
Paras	7	1.000	0.857	0.923
Parasect	6	1.000	1.000	1.000
Persian	6	0.857	1.000	0.923
Pidgeot	6	0.500	1.000	0.667
Pidgeotto	6	1.000	0.167	0.286
Pidgey	7	0.833	0.714	0.769
Pikachu	9	1.000	1.000	1.000
Pinsir	6	1.000	0.667	0.800
Poliwag	8	1.000	1.000	1.000
Poliwhirl	7	1.000	0.857	0.923
Poliwrath	6	0.833	0.833	0.833
Ponyta	7	0.833	0.714	0.769
Porygon	7	1.000	1.000	1.000
Primeape	8	0.750	0.750	0.750
Psyduck	6	1.000	0.833	0.909
Raichu	8	0.800	1.000	0.889
Rapidash	7	0.700	1.000	0.824
Raticate	6	1.000	1.000	1.000
Rattata	6	0.600	1.000	0.750
Rhydon	8	0.667	0.750	0.706
Rhyhorn	6	0.714	0.833	0.769
Sandshrew	7	1.000	1.000	1.000
Sandslash	7	1.000	1.000	1.000
Scyther	8	0.857	0.750	0.800
Seadra	7	1.000	0.714	0.833
Seaking	7	0.875	1.000	0.933
Seel	7	0.857	0.857	0.857
Shellder	7	1.000	0.714	0.833
Slowbro	7	1.000	0.714	0.833
Slowpoke	6	0.714	0.833	0.769
Snorlax	9	0.800	0.889	0.842
Spearow	8	0.833	0.625	0.714
Squirtle	8	1.000	1.000	1.000
Starmie	7	1.000	1.000	1.000
Staryu	7	1.000	1.000	1.000
Tangela	8	1.000	1.000	1.000
Tauros	8	0.889	1.000	0.941

Categoría	Soporte	Precisión	Recall	F1
Tentacool	7	0.875	1.000	0.933
Tentacruel	7	0.857	0.857	0.857
Vaporeon	9	0.889	0.889	0.889
Venomoth	8	1.000	0.875	0.933
Venonat	7	0.875	1.000	0.933
Venusaur	10	1.000	0.900	0.947
Victreebel	7	0.875	1.000	0.933
Vileplume	8	1.000	0.875	0.933
Voltorb	8	1.000	0.875	0.933
Vulpix	7	0.778	1.000	0.875
Wartortle	7	0.875	1.000	0.933
Weedle	7	1.000	0.857	0.923
Weepinbell	6	0.857	1.000	0.923
Weezing	8	0.778	0.875	0.824
Wigglytuff	8	1.000	0.875	0.933
Zapdos	8	1.000	0.875	0.933
Zubat	6	0.750	0.500	0.600